Group 7: Final Project Python Codebook

Names: Gianna Kim, Blair Warren, Djovan Velasco, Nathan Shindich, Stephanie Lei

1. Import Packages

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
import torch
from transformers import pipeline
import textwrap
from scipy import stats
import matplotlib.patches as mpatches
import warnings
warnings.filterwarnings('ignore')
```

2. Cleaning Data

(a) Renaming Long Variable Names

In [3]:
```
data = pd.read_csv('141XP Project Data.csv')
df = pd.DataFrame(data)

# Renaming Non-Likert Questions
df = df.rename(columns={
    'Timestamp': 'time',
    'Date that you took the survey': 'survey_date',
    'What are the last four digits of your University ID number?': 'la
    'Last name': 'last_name',
    'Course that you are enrolled in (Please answer in the format: dep
    'Your major': 'major',
    'Your minor': 'minor',
    'Gender identity ': 'gender',
    'If you prefer to self-describe your gender identity, please do so
    'Ethnic Background': 'ethnicity'
    })

# Renaming Likert Questions
df = df.rename(columns={
```

```
    'Are you first generation to attend college?': 'first_gen_college'
    "Level of mother's education ": 'mother_education_level',
    "Level of father's education ": 'father_education_level',
    'Are you a transfer student?': 'transfer_student',
    'Range of your GPA at UCLA': 'gpa_range',
    'How many community-engaged (XP) courses have you taken?': 'xp_cou
    'What motivated you to take an XP course?': 'xp_motivation',
    'If you are in a major or minor that requires completion of at lea
    'If you were to rate your sense of belonging (feeling included, he
    "I hesitate to speak up in my XP courses because I think my input
    'I feel respected by the other students on my project team.': 'res
    'I feel my perspective is taken seriously.': 'perspective_inclusio
    'I feel safe making mistakes while working with my project team.'
    'When final decisions are made, I feel my input is not taken into
    'I feel comfortable asking questions or requesting help when neede
    'I have a meaningful role in shaping the project team's decisions.
    "I worry my contributions aren't valuable.": 'non_valuable_contrib
    'I feel the community-engaged work is a meaningful way to contribu
    'I feel comfortable sharing my ideas.': 'comfortable_sharing_ideas
    'I feel ignored or dismissed by my project team.': 'feel_ignored',
    'I feel that the community partners treat our team as true collabo
    "The community partners' knowledge and experience help our team un
    'We don't interact enough with community partners to use their inp
    'The community partners' input meaningfully shapes the project's a
    'Scenario 1.\nAs part of an XP course, teams of two students are r
    'Explain the reason behind your choice in Scenario 1. Please be co
    'Scenario 2. \nAs part of an XP course, you worked with a communit
    'Explain the reason behind your choice in Scenario 2. Please be co
    'Scenario 3.\nThrough an XP course, you helped analyze data collec
    'Explain the reason behind your choice in Scenario 3. Please be co
    'Scenario 4. \nA friend of yours who is a graduating senior at UCL
    'Explain the reason behind your choice in Scenario 4. Please be co
    'Scenario 5.\nIn a community-engaged course, students are assigned
    'Explain the reason behind your choice in Scenario 5. Please be co
    })
```

(b) Conversion to Likert Scale

In [4]:
```
# ordered likert mapping
likert_map = {
    'Hardly ever': 1,
    'Occasionally': 2,
    'Sometimes': 3,
    'Often': 4,
    'Very often': 5
}

# each likert column
likert_cols = [
    'hesitation_to_participate',
    'respected_by_group',
```

```
    'perspective_inclusion',
    'mistake_safety_in_group',
    'input_not_considered',
    'comfort_asking_questions',
    'meaningful_role',
    'non_valuable_contribution',
    'community_work_valuable',
    'comfortable_sharing_ideas',
    'feel_ignored',
    'community_partners_inclusion',
    'community_partners_understanding',
    'lack_of_interaction_with_partners',
    'do_patners_help'
]

# map to each column
df[likert_cols] = df[likert_cols].apply(lambda col: col.map(likert_map

df[likert_cols].head()
```

Out[4]:

	hesitation_to_participate	respected_by_group	perspective_inclusion	mistak
0	4.0	5.0	NaN	
1	1.0	4.0	4.0	
2	1.0	5.0	5.0	
3	1.0	5.0	4.0	
4	2.0	5.0	5.0	

** Note: this creates the dataset 'cleaned_data.csv'*

3. Text Analysis

In [5]:
```
dat = pd.read_csv('cleaned_data.csv')
```

(a) Cleans Text Responses

In [6]:
```
import re

def clean_text(text):
    if pd.isna(text):
        return ""

    text = str(text)
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'http\S+|www\.\S+', '', text)
```

```
    text = text.replace('\n', ' ').replace('\r', ' ').replace('\t', '
    text = re.sub(r'\s+', ' ', text).strip()

    return text

reason_columns = [
    'scenario_1_reason',
    'scenario_2_reason',
    'scenario_3_reason',
    'scenario_4_reason',
    'scenario_5_reason'
]

for col in reason_columns:
    dat[f'{col}_clean'] = dat[col].apply(clean_text)

print(dat[[f'{col}_clean' for col in reason_columns]].head())
```

```
                            scenario_1_reason_clean  \
0  Before acting directly on it I would go to the...
1  I would definitely take a stand and respond to...
2  School is very important, but I would feel a s...
3  While it is important to acknowledge that acad...
4  I would tell my partner to explain how they ar...

                            scenario_2_reason_clean  \
0                I would be unsure what to say or do
1  I would ask the instructor because they probab...
2  I would like to get permission first to speak ...
3  Because the community hasn't asked me to speak...
4  if you want to speak up, you should help in an...

                            scenario_3_reason_clean  \
0                I would take action. Ask questions.
1  I would definitely consult the instructor. Giv...
2  The results don’t have to necessarily be publi...
3  Ethics, consent, and correct interpretation of...
4  the organization is who you are serving, so if...

                            scenario_4_reason_clean  \
0  I’m not sure if I read this correctly, but it ...
1  I would advise my friend to choose Job B. This...
2  I would advise my friend that they may feel mo...
3  I think the experience of working with both in...
4  More opportunities to expand the scope of the ...

                            scenario_5_reason_clean
0  I think a lot of barriers come up around findi...
1  I would say move the workshops to zoom. This w...
2  In-person meetings are much more valuable ways...
3  Zoom meetings are more convenient than in-pers...
4  Offer a hybrid option where families that cann...
```

(b) Sentiment Analysis

In [7]:
```
from transformers import pipeline
from tqdm import tqdm
tqdm.pandas()

sentiment_analyzer = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-roberta-base-sentiment-latest",
    truncation=True,
    max_length=512
)
```

```
Some weights of the model checkpoint at cardiffnlp/twitter-roberta-base
-sentiment-latest were not used when initializing RobertaForSequenceCla
ssification: ['roberta.pooler.dense.bias', 'roberta.pooler.dense.weigh
t']
- This IS expected if you are initializing RobertaForSequenceClassifica
tion from the checkpoint of a model trained on another task or with ano
ther architecture (e.g. initializing a BertForSequenceClassification mo
del from a BertForPreTraining model).
- This IS NOT expected if you are initializing RobertaForSequenceClassi
fication from the checkpoint of a model that you expect to be exactly i
dentical (initializing a BertForSequenceClassification model from a Ber
tForSequenceClassification model).
Hardware accelerator e.g. GPU is available in the environment, but no `
device` argument is passed to the `Pipeline` object. Model will be on C
PU.
```

(c) Creating Sentiment Dataset

In [8]:
```
def get_continuous_sentiment(text):
    if not text or pd.isna(text) or text.strip() == "":
        return None
    try:
        result = sentiment_analyzer(text)[0]
        label = result['label'].lower()
        score = result['score']

        if label == 'positive':
            return score
        elif label == 'negative':
            return -score
        else:
            return 0.0

    except Exception as e:
        return None

clean_columns = [
    'scenario_1_reason_clean',
```

```
    'scenario_2_reason_clean',
    'scenario_3_reason_clean',
    'scenario_4_reason_clean',
    'scenario_5_reason_clean'
]

for col in clean_columns:
    new_col_name = col.replace('_clean', '_sentiment')
    print(f"Processing {col}...")
    dat[new_col_name] = dat[col].progress_apply(get_continuous_sentime

sentiment_columns = [col.replace('_clean', '_sentiment') for col in cl
dat['overall_scenario_sentiment'] = dat[sentiment_columns].mean(axis=1
```

```
Processing scenario_1_reason_clean...
100%|██████████| 45/45 [00:03<00:00, 14.12it/s]
Processing scenario_2_reason_clean...
100%|██████████| 45/45 [00:02<00:00, 15.74it/s]
Processing scenario_3_reason_clean...
100%|██████████| 45/45 [00:02<00:00, 20.60it/s]
Processing scenario_4_reason_clean...
100%|██████████| 45/45 [00:01<00:00, 23.49it/s]
Processing scenario_5_reason_clean...
100%|██████████| 45/45 [00:02<00:00, 17.90it/s]
```

In [9]:
```
dat.head()
```

Out[9]:

	time	survey_date	last_4_digits_uid	last_name	enrolled_course	maj
0	1/29/2026 15:02:43	1/29/2026	532	Whitney	ELTS120XP	Glob studi
1	1/29/2026 22:54:03	1/29/2026	7824	Sleeper	ENGCOMP130DX	Pub Affa
2	1/30/2026 12:26:40	1/30/2026	3402	Owen	CESC 191AX	Politi Scien
3	1/30/2026 13:24:01	1/30/2026	4689	Wenn	CESC191AX	Stu Religi
4	1/30/2026 13:30:49	1/30/2026	2809	Sobel	CESC 191AX	Politi Scien

5 rows × 55 columns

** Note: this creates the dataset 'dat_withSentiment.csv'*

(d) Reshaping Sentiment Dataset

In [10]:
```
# Assuming your dataframe is named 'df' and you have already generated
sentiment_cols = [
    'scenario_1_reason_sentiment', 'scenario_2_reason_sentiment',
    'scenario_3_reason_sentiment', 'scenario_4_reason_sentiment',
    'scenario_5_reason_sentiment'
]

# Variables you want to keep for grouping/x-axis
metadata_cols = ['enrolled_course', 'major', 'minor', 'gender', 'gende
       'ethnicity', 'first_gen_college', 'mother_education_level',
       'father_education_level', 'transfer_student', 'gpa_range', 'xp_
       'xp_motivation', 'major_minor_motivation', 'belonging_rate',
       'hesitation_to_participate', 'respected_by_group',
       'perspective_inclusion', 'mistake_safety_in_group',
       'input_not_considered', 'comfort_asking_questions', 'meaningful
       'non_valuable_contribution', 'community_work_valuable',
       'comfortable_sharing_ideas', 'feel_ignored',
       'community_partners_inclusion', 'community_partners_understandi
       'lack_of_interaction_with_partners', 'do_patners_help', 'scenar
       'scenario_1_reason', 'scenario_2', 'scenario_2_reason', 'scenar
       'scenario_3_reason', 'scenario_4', 'scenario_4_reason', 'scenar
       'scenario_5_reason']
```

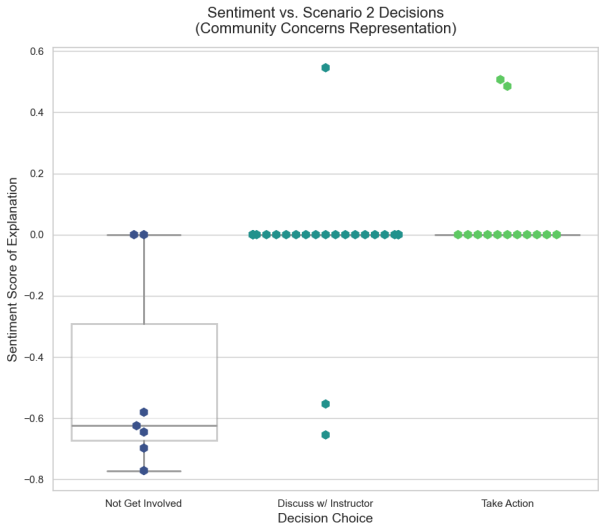
```
# Reshape the data
df_long = pd.melt(
    dat,
    id_vars=metadata_cols,
    value_vars=sentiment_cols,
    var_name='Scenario',
    value_name='Sentiment_Score'
)

# Clean up the 'Scenario' names for the legend (e.g., "scenario_1_sent
df_long['Scenario'] = df_long['Scenario'].str.replace('_sentiment', ''
```

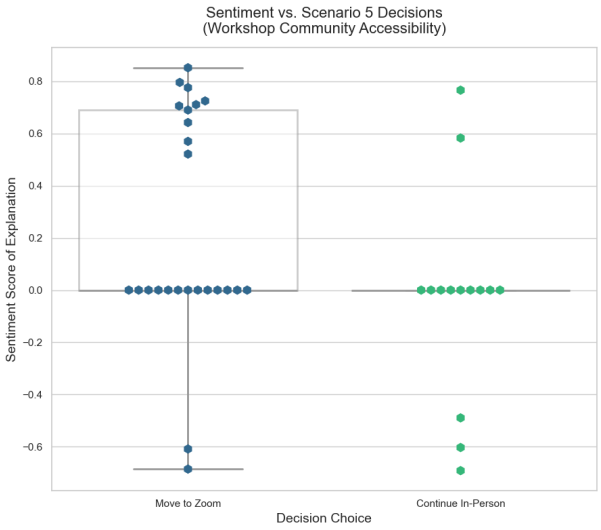
4. Research Question 1

In [28]:

```
# Load data
df = pd.read_csv('dat_withSentiment.csv')

# Connection heuristic items
pos_items = [
    'respected_by_group', 'perspective_inclusion', 'mistake_safety_in_
    'comfort_asking_questions', 'meaningful_role', 'community_work_val
    'comfortable_sharing_ideas', 'community_partners_inclusion',
    'community_partners_understanding', 'do_patners_help'
]

rev_items = [
    'hesitation_to_participate', 'input_not_considered',
    'non_valuable_contribution', 'feel_ignored',
    'lack_of_interaction_with_partners'
]

for item in rev_items:
    df[item + '_rev'] = 6 - df[item]

all_heuristic_items = pos_items + [item + '_rev' for item in rev_items
df['Connection_Score'] = df[all_heuristic_items].mean(axis=1)

sns.set_theme(style="whitegrid")

# Plot 1
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

# 1st plot: Barplot
item_means = df[all_heuristic_items].mean().sort_values(ascending=True
item_names = [name.replace('_rev', ' (Reversed)').replace('_', ' ').ti

sns.barplot(x=item_means.values, y=item_names, ax=axes[0], palette="vi
axes[0].set_title('Mean Scores for Connection Heuristic Items', fontsi
axes[0].set_xlabel('Average Score (1 to 5)', fontsize=14)
axes[0].set_xlim(0, 5)
```

```
# 2nd plot: Swarm plot (hex-grid like points) for distribution instead
sns.swarmplot(y=df['Connection_Score'], ax=axes[1], palette="viridis",
# Also add a boxplot with stepped lines or just keep the swarm
sns.boxplot(y=df['Connection_Score'], ax=axes[1], color="white", linew
axes[1].set_title('Distribution of Overall Connection Heuristic Score'
axes[1].set_ylabel('Connection Score (1 to 5)', fontsize=14)
axes[1].set_ylim(2.8, 5.2)

plt.tight_layout()
plt.savefig('connection_heuristic_v3.png', dpi=300)
plt.close()


# Plot 2
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

df['scenario_2_short'] = df['scenario_2'].map({
    'I would first discuss the situation with my instructor or the com
    'I would take some form of action to ensure the community's concer
    'Given the circumstances, it would be better not to get involved o
})

# Use swarmplot with hex markers instead of smooth violins
sns.boxplot(x='scenario_2_short', y='scenario_2_reason_sentiment', dat
            color='white', linewidth=2, fliersize=0, boxprops=dict(alp
sns.swarmplot(x='scenario_2_short', y='scenario_2_reason_sentiment', d
              palette='viridis', marker='h', size=10, hue='scenario_2_
axes[0].set_title('Sentiment vs. Scenario 2 Decisions\n(Community Conc
axes[0].set_xlabel('Decision Choice', fontsize=14)
axes[0].set_ylabel('Sentiment Score of Explanation', fontsize=14)

df['scenario_5_short'] = df['scenario_5'].map({
    'Move to zoom workshops': 'Move to Zoom',
    'Continue with in-person workshops': 'Continue In-Person'
})

sns.boxplot(x='scenario_5_short', y='scenario_5_reason_sentiment', dat
            color='white', linewidth=2, fliersize=0, boxprops=dict(alp
sns.swarmplot(x='scenario_5_short', y='scenario_5_reason_sentiment', d
              palette='viridis', marker='h', size=10, hue='scenario_5_
axes[1].set_title('Sentiment vs. Scenario 5 Decisions\n(Workshop Commu
axes[1].set_xlabel('Decision Choice', fontsize=14)
axes[1].set_ylabel('Sentiment Score of Explanation', fontsize=14)

plt.tight_layout()
plt.savefig('scenario_sentiments_v3.png', dpi=300)
plt.show()
plt.close()
```

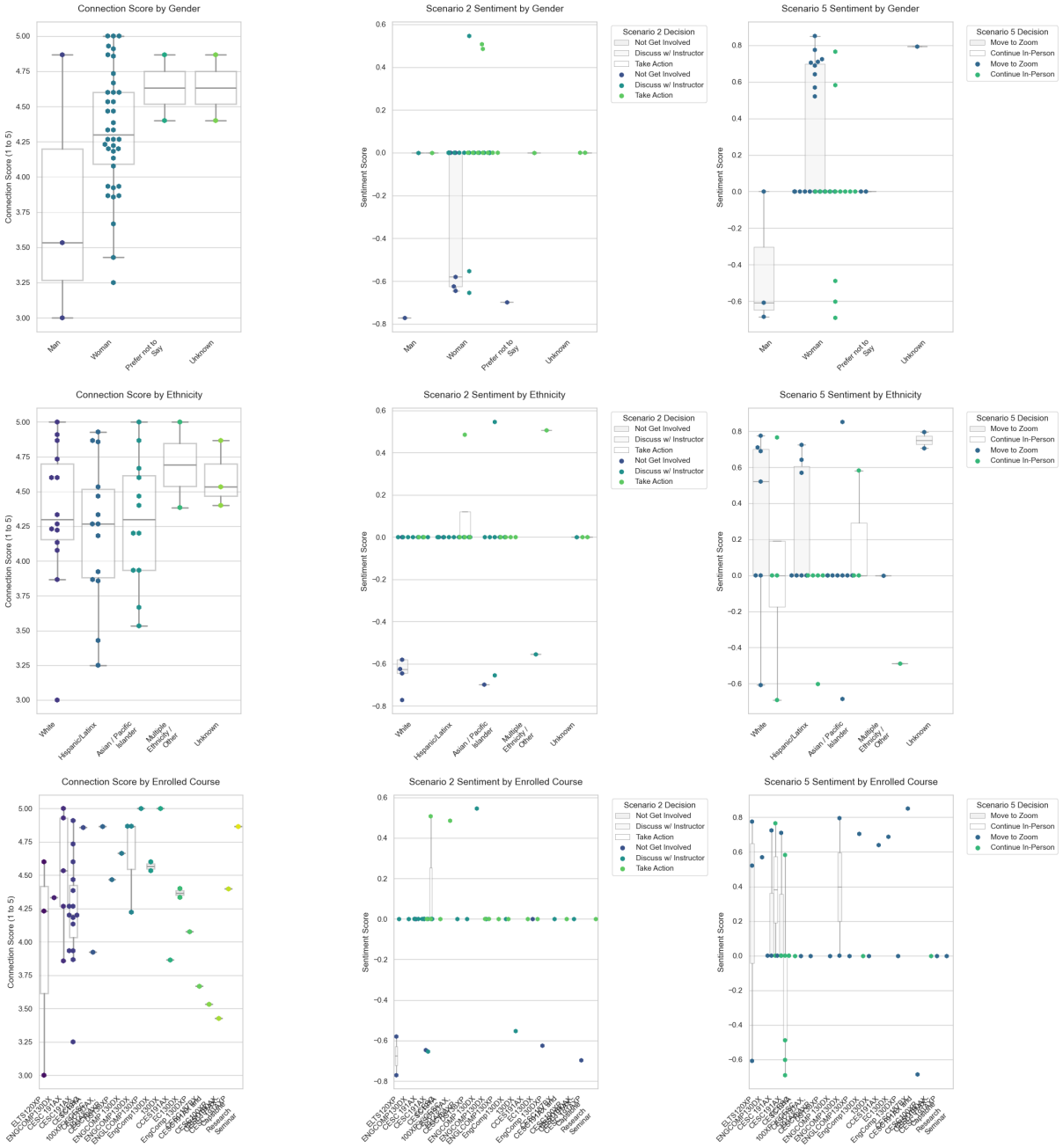




In [11]:
```
# Load data
df = pd.read_csv('dat_withSentiment.csv')

# Connection heuristic items
pos_items = [
    'respected_by_group', 'perspective_inclusion', 'mistake_safety_in_
    'comfort_asking_questions', 'meaningful_role', 'community_work_val
    'comfortable_sharing_ideas', 'community_partners_inclusion',
    'community_partners_understanding', 'do_patners_help'
]

rev_items = [
    'hesitation_to_participate', 'input_not_considered',
    'non_valuable_contribution', 'feel_ignored',
    'lack_of_interaction_with_partners'
]

for item in rev_items:
    df[item + '_rev'] = 6 - df[item]

all_heuristic_items = pos_items + [item + '_rev' for item in rev_items
df['Connection_Score'] = df[all_heuristic_items].mean(axis=1)

# Map Scenarios
df['scenario_2_short'] = df['scenario_2'].map({
    'I would first discuss the situation with my instructor or the com
    'I would take some form of action to ensure the community's concer
    'Given the circumstances, it would be better not to get involved o
})

df['scenario_5_short'] = df['scenario_5'].map({
    'Move to zoom workshops': 'Move to Zoom',
    'Continue with in-person workshops': 'Continue In-Person'
})

sns.set_theme(style="whitegrid")
```

```
# Helper function to wrap text for labels
def wrap_labels(ax, width, break_long_words=False):
    labels = []
    for label in ax.get_xticklabels():
        text = label.get_text()
        labels.append(textwrap.fill(text, width=width, break_long_word
    ax.set_xticklabels(labels, rotation=45, ha='right')

facet_vars = ['gender', 'ethnicity', 'enrolled_course']
titles = ['Gender', 'Ethnicity', 'Enrolled Course']

for var, title in zip(facet_vars, titles):
    # Fill NaN with 'Unknown' for plotting
    df[var] = df[var].fillna('Unknown')

    fig, axes = plt.subplots(1, 3, figsize=(22, 8))

    # 1. Connection Score Distribution
    sns.boxplot(x=var, y='Connection_Score', data=df, ax=axes[0],
                color='white', linewidth=2, fliersize=0, boxprops=dict
    sns.swarmplot(x=var, y='Connection_Score', data=df, ax=axes[0],
                  palette='viridis', marker='h', size=8, hue=var, lege
    axes[0].set_title(f'Connection Score by {title}', fontsize=14, pad
    axes[0].set_ylabel('Connection Score (1 to 5)')
    axes[0].set_xlabel('')
    wrap_labels(axes[0], 15)

    # 2. Scenario 2
    sns.boxplot(x=var, y='scenario_2_reason_sentiment', hue='scenario_
                color='white', linewidth=1, fliersize=0, boxprops=dict
    sns.swarmplot(x=var, y='scenario_2_reason_sentiment', hue='scenari
                  palette='viridis', marker='h', size=7, dodge=True)
    axes[1].set_title(f'Scenario 2 Sentiment by {title}', fontsize=14,
    axes[1].set_ylabel('Sentiment Score')
    axes[1].set_xlabel('')
    axes[1].legend(title='Scenario 2 Decision', bbox_to_anchor=(1.05,
    wrap_labels(axes[1], 15)

    # 3. Scenario 5
    sns.boxplot(x=var, y='scenario_5_reason_sentiment', hue='scenario_
                color='white', linewidth=1, fliersize=0, boxprops=dict
    sns.swarmplot(x=var, y='scenario_5_reason_sentiment', hue='scenari
                  palette='viridis', marker='h', size=7, dodge=True)
    axes[2].set_title(f'Scenario 5 Sentiment by {title}', fontsize=14,
    axes[2].set_ylabel('Sentiment Score')
    axes[2].set_xlabel('')
    axes[2].legend(title='Scenario 5 Decision', bbox_to_anchor=(1.05,
    wrap_labels(axes[2], 15)

    plt.tight_layout()
    plt.savefig(f'faceted_{var}.png', dpi=300)
    plt.show()
```

```
    plt.close()

print("Faceted plots generated successfully.")
```



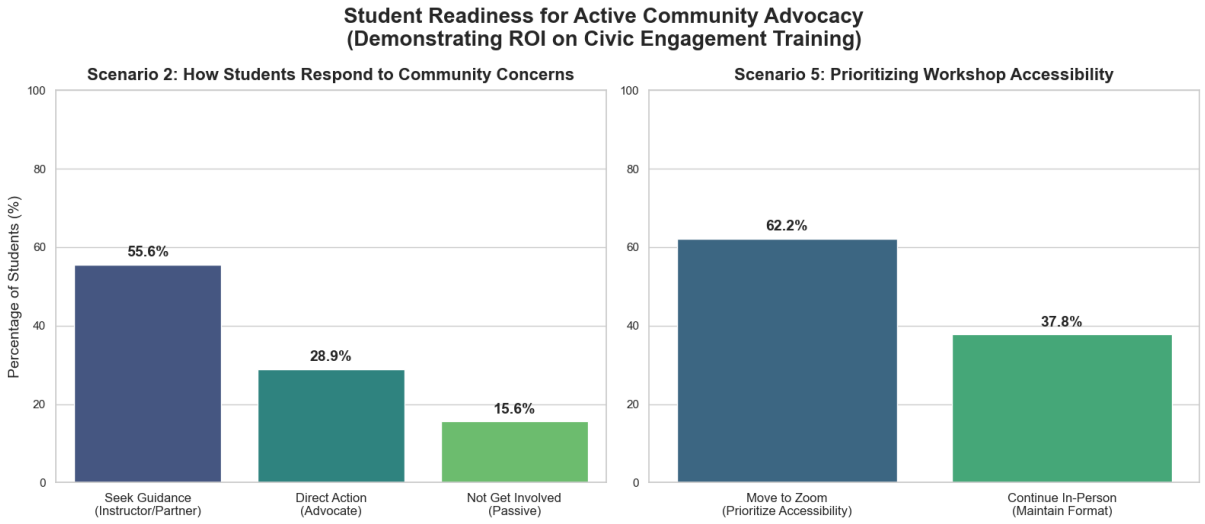
Faceted plots generated successfully.

In [30]:
```
# Connection heuristic items
pos_items = [
    'respected_by_group', 'perspective_inclusion', 'mistake_safety_in_
    'comfort_asking_questions', 'meaningful_role', 'community_work_val
    'comfortable_sharing_ideas', 'community_partners_inclusion',
    'community_partners_understanding', 'do_patners_help'
]

rev_items = [
    'hesitation_to_participate', 'input_not_considered',
    'non_valuable_contribution', 'feel_ignored',
```

```
    'lack_of_interaction_with_partners'
]

for item in rev_items:
    df[item + '_rev'] = 6 - df[item]

all_heuristic_items = pos_items + [item + '_rev' for item in rev_items
df['Connection_Score'] = df[all_heuristic_items].mean(axis=1)

sns.set_theme(style="whitegrid")

# Figure 1: Course Engagement (Resource Allocation focus)
fig, ax = plt.subplots(figsize=(14, 7))
course_avg = df.groupby('enrolled_course')['Connection_Score'].mean().

# Clean course names for display if needed
course_labels = [textwrap.fill(str(c), 25) for c in course_avg.index]

sns.barplot(x=course_avg.values, y=course_labels, palette="viridis", a
ax.set_title('Average Community Connection Score by Course\n(Identifyi
ax.set_xlabel('Average Connection Score (Scale: 1 to 5)', fontsize=14)
ax.set_ylabel('')
ax.set_xlim(3.0, 5.0) # Zooming in on the 3-5 range since all scores a

# Add value labels
for p in ax.patches:
    ax.annotate(f"{p.get_width():.2f}",
                (p.get_width() + 0.03, p.get_y() + p.get_height() / 2.
                ha='left', va='center', fontsize=12, color='black', we

plt.tight_layout()
plt.savefig('course_resource_allocation.png', dpi=300)
plt.close()


# Figure 2: Equity in Engagement (Demographics Focus)
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# First-Gen
fg_avg = df.groupby('first_gen_college')['Connection_Score'].mean().dr
sns.barplot(x=fg_avg.index, y=fg_avg.values, palette="viridis", ax=axe
axes[0].set_title('Connection by First-Generation Status\n(Targeting s
axes[0].set_ylabel('Average Connection Score (Scale: 1 to 5)', fontsiz
axes[0].set_xlabel('First Generation College Student?', fontsize=14)
axes[0].set_ylim(3.0, 5.0)

for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height():.2f}",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

# Ethnicity
```

```
eth_avg = df.groupby('ethnicity')['Connection_Score'].mean().dropna().
eth_labels = [textwrap.fill(str(l), 15) for l in eth_avg.index]

sns.barplot(x=eth_labels, y=eth_avg.values, palette="viridis", ax=axes
axes[1].set_title('Connection by Ethnicity\n(Identifying equity gaps a
axes[1].set_ylabel('')
axes[1].set_xlabel('')
axes[1].set_ylim(3.0, 5.0)
axes[1].tick_params(axis='x', labelsize=12)

for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.2f}",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

plt.suptitle('Equity and Inclusion: Where Should Targeted Resources Go
plt.tight_layout()
plt.savefig('equity_resource_allocation.png', dpi=300)
plt.close()


# Figure 3: Scenario Action Propensity (Civic Responsibility Focus)
# Simplifying scenario outcomes to binary/ternary categories
df['scen2_action'] = df['scenario_2'].map({
    'I would first discuss the situation with my instructor or the com
    'I would take some form of action to ensure the community's concer
    'Given the circumstances, it would be better not to get involved o
})
scen2_counts = df['scen2_action'].value_counts(normalize=True) * 100

df['scen5_action'] = df['scenario_5'].map({
    'Move to zoom workshops': 'Move to Zoom\n(Prioritize Accessibility
    'Continue with in-person workshops': 'Continue In-Person\n(Maintai
})
scen5_counts = df['scen5_action'].value_counts(normalize=True) * 100

fig, axes = plt.subplots(1, 2, figsize=(16, 7))

sns.barplot(x=scen2_counts.index, y=scen2_counts.values, palette="viri
axes[0].set_title('Scenario 2: How Students Respond to Community Conce
axes[0].set_ylabel('Percentage of Students (%)', fontsize=14)
axes[0].set_xlabel('')
axes[0].set_ylim(0, 100)
axes[0].tick_params(axis='x', labelsize=12)

for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height():.1f}%",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

sns.barplot(x=scen5_counts.index, y=scen5_counts.values, palette="viri
axes[1].set_title('Scenario 5: Prioritizing Workshop Accessibility', f
```

```
axes[1].set_ylabel('')
axes[1].set_xlabel('')
axes[1].set_ylim(0, 100)
axes[1].tick_params(axis='x', labelsize=12)

for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.1f}%",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

plt.suptitle('Student Readiness for Active Community Advocacy\n(Demons
plt.tight_layout()
plt.savefig('advocacy_readiness.png', dpi=300)
plt.show()
plt.close()
```



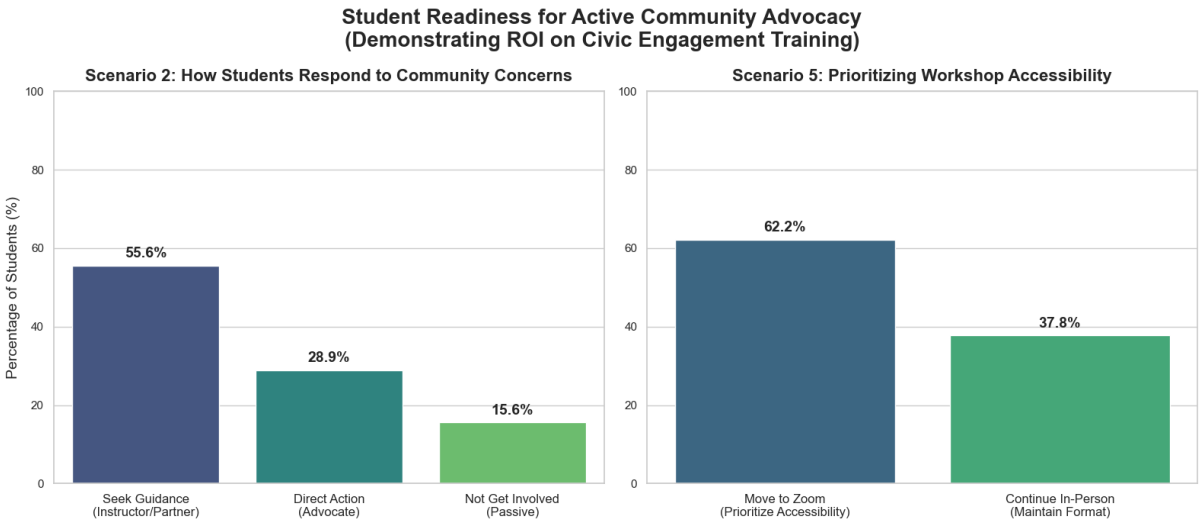
In [32]:

```
# Connection heuristic items
pos_items = [
    'respected_by_group', 'perspective_inclusion', 'mistake_safety_in_
    'comfort_asking_questions', 'meaningful_role', 'community_work_val
    'comfortable_sharing_ideas', 'community_partners_inclusion',
    'community_partners_understanding', 'do_patners_help'
]

rev_items = [
    'hesitation_to_participate', 'input_not_considered',
    'non_valuable_contribution', 'feel_ignored',
    'lack_of_interaction_with_partners'
]

for item in rev_items:
    df[item + '_rev'] = 6 - df[item]

all_heuristic_items = pos_items + [item + '_rev' for item in rev_items
df['Connection_Score'] = df[all_heuristic_items].mean(axis=1)

sns.set_theme(style="whitegrid")
```

```
# Figure 1: Course Engagement (Resource Allocation focus)
fig, ax = plt.subplots(figsize=(14, 7))
course_avg = df.groupby('enrolled_course')['Connection_Score'].mean().

# Clean course names for display if needed
course_labels = [textwrap.fill(str(c), 25) for c in course_avg.index]

sns.barplot(x=course_avg.values, y=course_labels, palette="viridis", a
ax.set_title('Average Community Connection Score by Course\n(Identifyi
ax.set_xlabel('Average Connection Score (Scale: 1 to 5)', fontsize=14)
ax.set_ylabel('')
ax.set_xlim(3.0, 5.0) # Zooming in on the 3-5 range since all scores a

# Add value labels
for p in ax.patches:
    ax.annotate(f"{p.get_width():.2f}",
                (p.get_width() + 0.03, p.get_y() + p.get_height() / 2.
                ha='left', va='center', fontsize=12, color='black', we

plt.tight_layout()
plt.savefig('course_resource_allocation.png', dpi=300)
plt.close()


# Figure 2: Equity in Engagement (Demographics Focus)
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# First-Gen
fg_avg = df.groupby('first_gen_college')['Connection_Score'].mean().dr
sns.barplot(x=fg_avg.index, y=fg_avg.values, palette="viridis", ax=axe
axes[0].set_title('Connection by First-Generation Status\n(Targeting s
axes[0].set_ylabel('Average Connection Score (Scale: 1 to 5)', fontsiz
axes[0].set_xlabel('First Generation College Student?', fontsize=14)
axes[0].set_ylim(3.0, 5.0)

for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height():.2f}",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

# Ethnicity
eth_avg = df.groupby('ethnicity')['Connection_Score'].mean().dropna().
eth_labels = [textwrap.fill(str(l), 15) for l in eth_avg.index]

sns.barplot(x=eth_labels, y=eth_avg.values, palette="viridis", ax=axes
axes[1].set_title('Connection by Ethnicity\n(Identifying equity gaps a
axes[1].set_ylabel('')
axes[1].set_xlabel('')
axes[1].set_ylim(3.0, 5.0)
axes[1].tick_params(axis='x', labelsize=12)
```

```
for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.2f}",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

plt.suptitle('Equity and Inclusion: Where Should Targeted Resources Go
plt.tight_layout()
plt.savefig('equity_resource_allocation.png', dpi=300)
plt.close()


# Figure 3: Scenario Action Propensity (Civic Responsibility Focus)
# Simplifying scenario outcomes to binary/ternary categories
df['scen2_action'] = df['scenario_2'].map({
    'I would first discuss the situation with my instructor or the com
    'I would take some form of action to ensure the community's concer
    'Given the circumstances, it would be better not to get involved o
})
scen2_counts = df['scen2_action'].value_counts(normalize=True) * 100

df['scen5_action'] = df['scenario_5'].map({
    'Move to zoom workshops': 'Move to Zoom\n(Prioritize Accessibility
    'Continue with in-person workshops': 'Continue In-Person\n(Maintai
})
scen5_counts = df['scen5_action'].value_counts(normalize=True) * 100

fig, axes = plt.subplots(1, 2, figsize=(16, 7))

sns.barplot(x=scen2_counts.index, y=scen2_counts.values, palette="viri
axes[0].set_title('Scenario 2: How Students Respond to Community Conce
axes[0].set_ylabel('Percentage of Students (%)', fontsize=14)
axes[0].set_xlabel('')
axes[0].set_ylim(0, 100)
axes[0].tick_params(axis='x', labelsize=12)

for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height():.1f}%",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo

sns.barplot(x=scen5_counts.index, y=scen5_counts.values, palette="viri
axes[1].set_title('Scenario 5: Prioritizing Workshop Accessibility', f
axes[1].set_ylabel('')
axes[1].set_xlabel('')
axes[1].set_ylim(0, 100)
axes[1].tick_params(axis='x', labelsize=12)

for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.1f}%",
                     (p.get_x() + p.get_width() / 2., p.get_height() +
                     ha='center', va='center', fontsize=14, weight='bo
```

```
plt.suptitle('Student Readiness for Active Community Advocacy\n(Demons
plt.tight_layout()
plt.savefig('advocacy_readiness.png', dpi=300)
plt.show()
plt.close()
```



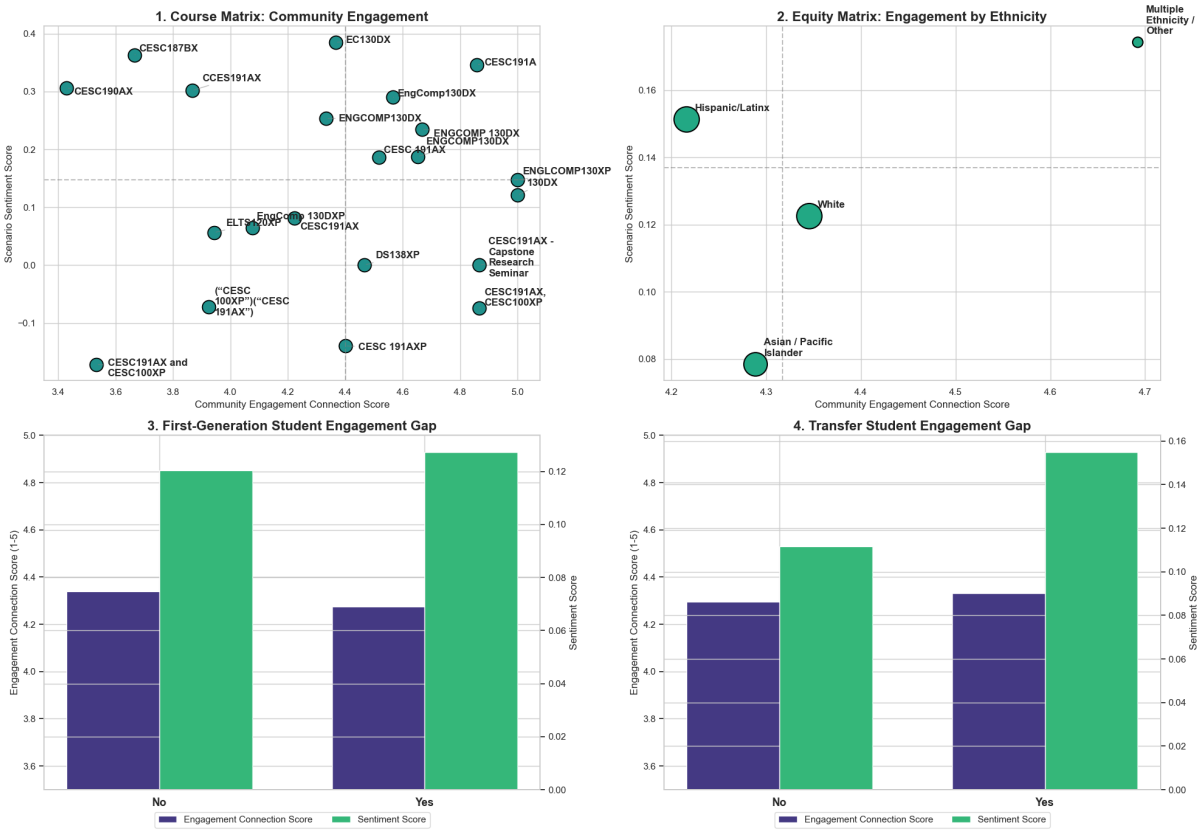
In [34]:

```
import random

# Connection heuristic items explicitly mapped to Community Engagement
pos_items = [
    'respected_by_group', 'perspective_inclusion', 'mistake_safety_in_
    'comfort_asking_questions', 'meaningful_role', 'community_work_val
    'comfortable_sharing_ideas', 'community_partners_inclusion',
    'community_partners_understanding', 'do_patners_help'
]
rev_items = [
    'hesitation_to_participate', 'input_not_considered',
    'non_valuable_contribution', 'feel_ignored',
    'lack_of_interaction_with_partners'
]

for item in rev_items:
    df[item + '_rev'] = 6 - df[item]

all_heuristic_items = pos_items + [item + '_rev' for item in rev_items
df['CE_Connection_Score'] = df[all_heuristic_items].mean(axis=1)

# Ensure overall sentiment is calculated
sentiment_cols = [c for c in df.columns if 'sentiment' in c and c != '
df['overall_scenario_sentiment'] = df[sentiment_cols].mean(axis=1)

sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(20, 16))

# Plot 1: Course Quadrant
ax1 = axes[0, 0]
course_data = df.groupby('enrolled_course')[['CE_Connection_Score', 'o
```

```
sns.scatterplot(data=course_data, x='CE_Connection_Score', y='overall_
                s=250, color=sns.color_palette("viridis", 1)[0], ax=ax

# Quadrant lines
med_conn_course = course_data['CE_Connection_Score'].median()
med_sent_course = course_data['overall_scenario_sentiment'].median()
ax1.axvline(med_conn_course, color='gray', linestyle='--', alpha=0.5)
ax1.axhline(med_sent_course, color='gray', linestyle='--', alpha=0.5)

# Annotations
for course in course_data.index:
    # Generate a random offset for each label
    x_offset = random.randint(5, 15)
    y_offset = random.randint(-15, 15)

    ax1.annotate(textwrap.fill(str(course), 15),
                 (course_data.loc[course, 'CE_Connection_Score'], cour
                 xytext=(x_offset, y_offset),
                 textcoords='offset points',
                 fontsize=12,
                 weight='bold',
                 arrowprops=dict(arrowstyle="-", color='gray', alpha=0

ax1.set_title('1. Course Matrix: Community Engagement', fontsize=16, w
ax1.set_xlabel('Community Engagement Connection Score', fontsize=12)
ax1.set_ylabel('Scenario Sentiment Score', fontsize=12)

# Plot 2: Ethnicity Quadrant
ax2 = axes[0, 1]
eth_data = df.groupby('ethnicity')[['CE_Connection_Score', 'overall_sc
# Add count for bubble size
eth_data['count'] = df['ethnicity'].value_counts()

sns.scatterplot(data=eth_data, x='CE_Connection_Score', y='overall_sce
                size='count', sizes=(150, 900), color=sns.color_palett

med_conn_eth = eth_data['CE_Connection_Score'].median()
med_sent_eth = eth_data['overall_scenario_sentiment'].median()
ax2.axvline(med_conn_eth, color='gray', linestyle='--', alpha=0.5)
ax2.axhline(med_sent_eth, color='gray', linestyle='--', alpha=0.5)

for eth in eth_data.index:
    ax2.annotate(textwrap.fill(str(eth), 15),
                 (eth_data.loc[eth, 'CE_Connection_Score'], eth_data.l
                 xytext=(10, 10), textcoords='offset points', fontsize

ax2.set_title('2. Equity Matrix: Engagement by Ethnicity', fontsize=16
ax2.set_xlabel('Community Engagement Connection Score', fontsize=12)
ax2.set_ylabel('Scenario Sentiment Score', fontsize=12)

# Plot 3: First-Gen Bar Chart (Dual Axis)
```

```
ax3 = axes[1, 0]
fg_data = df.groupby('first_gen_college')[['CE_Connection_Score', 'ove

x = np.arange(len(fg_data.index))
width = 0.35

ax3_twin = ax3.twinx()
bar1 = ax3.bar(x - width/2, fg_data['CE_Connection_Score'], width, lab
bar2 = ax3_twin.bar(x + width/2, fg_data['overall_scenario_sentiment']

ax3.set_ylabel('Engagement Connection Score (1-5)', fontsize=12)
ax3_twin.set_ylabel('Sentiment Score', fontsize=12)
ax3.set_xticks(x)
ax3.set_xticklabels(fg_data.index, fontsize=13, weight='bold')
ax3.set_title('3. First-Generation Student Engagement Gap', fontsize=1

lines, labels = ax3.get_legend_handles_labels()
lines2, labels2 = ax3_twin.get_legend_handles_labels()
ax3_twin.legend(lines + lines2, labels + labels2, loc='upper center',
ax3.set_ylim(3.5, 5.0)

# Plot 4: Transfer Student Bar Chart (Dual Axis)
ax4 = axes[1, 1]
ts_data = df.groupby('transfer_student')[['CE_Connection_Score', 'over

x2 = np.arange(len(ts_data.index))

ax4_twin = ax4.twinx()
bar3 = ax4.bar(x2 - width/2, ts_data['CE_Connection_Score'], width, la
bar4 = ax4_twin.bar(x2 + width/2, ts_data['overall_scenario_sentiment'

ax4.set_ylabel('Engagement Connection Score (1-5)', fontsize=12)
ax4_twin.set_ylabel('Sentiment Score', fontsize=12)
ax4.set_xticks(x2)
ax4.set_xticklabels(ts_data.index, fontsize=13, weight='bold')
ax4.set_title('4. Transfer Student Engagement Gap', fontsize=16, weigh

lines3, labels3 = ax4.get_legend_handles_labels()
lines4, labels4 = ax4_twin.get_legend_handles_labels()
ax4_twin.legend(lines3 + lines4, labels3 + labels4, loc='upper center'
ax4.set_ylim(3.5, 5.0)

fig.suptitle("Student Sentiment against Core Engagement Metrics (Belon

plt.tight_layout(rect=[0, 0.03, 1, 0.93])
plt.savefig('community_engagement_resource_allocation.png', dpi=300)
plt.show()
plt.close()
```

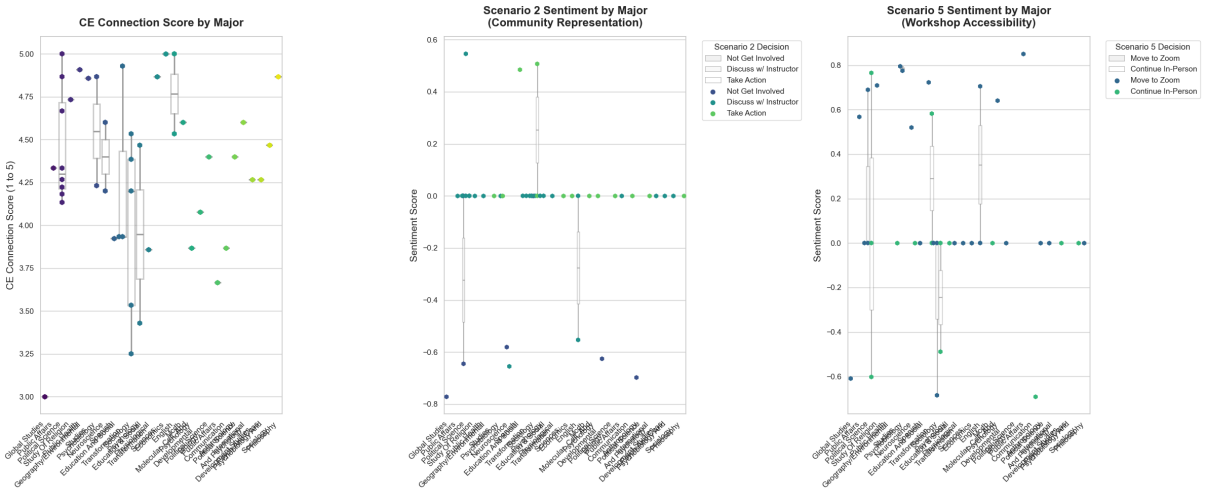
Student Sentiment against Core Engagement Metrics (Belonging, Meaningful Roles, Partner Collaboration)



In [37]:
```
# Connection heuristic items explicitly mapped to Community Engagement
pos_items = [
    'respected_by_group', 'perspective_inclusion', 'mistake_safety_in_
    'comfort_asking_questions', 'meaningful_role', 'community_work_val
    'comfortable_sharing_ideas', 'community_partners_inclusion',
    'community_partners_understanding', 'do_patners_help'
]
rev_items = [
    'hesitation_to_participate', 'input_not_considered',
    'non_valuable_contribution', 'feel_ignored',
    'lack_of_interaction_with_partners'
]

# Reverse the negative items (assuming 1–5 scale)
for item in rev_items:
    df[item + '_rev'] = 6 - df[item]

all_heuristic_items = pos_items + [item + '_rev' for item in rev_items
df['CE_Connection_Score'] = df[all_heuristic_items].mean(axis=1)

# Clean up major names and handle missing values
df['major'] = df['major'].fillna('Unknown')
df['major'] = df['major'].str.title()

# Map Scenarios to shorter labels
df['scenario_2_short'] = df['scenario_2'].map({
```

```
    'I would first discuss the situation with my instructor or the com
    'I would take some form of action to ensure the community's concer
    'Given the circumstances, it would be better not to get involved o
})

df['scenario_5_short'] = df['scenario_5'].map({
    'Move to zoom workshops': 'Move to Zoom',
    'Continue with in-person workshops': 'Continue In-Person'
})

sns.set_theme(style="whitegrid")

# Helper function to wrap text
def wrap_labels(ax, width, break_long_words=False):
    labels = []
    for label in ax.get_xticklabels():
        text = label.get_text()
        labels.append(textwrap.fill(text, width=width, break_long_word
    ax.set_xticklabels(labels, rotation=45, ha='right')

# --- Plot 1: Horizontal Bar Chart for Resource Allocation by Major --
fig, ax = plt.subplots(figsize=(14, 10))  # Correctly uses plt.subplot
major_data = df.groupby('major')['CE_Connection_Score'].mean().dropna(

# Clean major names for display
major_labels = [textwrap.fill(str(m), 30) for m in major_data.index]

sns.barplot(x=major_data.values, y=major_labels, palette="viridis", ax
ax.set_title('Average Community Engagement Connection Score by Major',
ax.set_xlabel('Average CE Connection Score (Scale: 1 to 5)', fontsize=
ax.set_ylabel('')
ax.set_xlim(3.0, 5.0)

for p in ax.patches:
    ax.annotate(f"{p.get_width():.2f}",
                (p.get_width() + 0.02, p.get_y() + p.get_height() / 2.
                ha='left', va='center', fontsize=11, weight='bold')

plt.tight_layout()
plt.savefig('major_resource_bar.png', dpi=300)
plt.close()

# --- Plot 2: Faceted Distribution Plot by Major ---
fig, axes = plt.subplots(1, 3, figsize=(24, 10)) # Correctly uses plt.

# 1. CE Connection Score Distribution
sns.boxplot(x='major', y='CE_Connection_Score', data=df, ax=axes[0],
            color='white', linewidth=2, fliersize=0, boxprops=dict(alp
sns.swarmplot(x='major', y='CE_Connection_Score', data=df, ax=axes[0],
              palette='viridis', marker='h', size=8, hue='major', lege
axes[0].set_title('CE Connection Score by Major', fontsize=16, pad=15,
axes[0].set_ylabel('CE Connection Score (1 to 5)', fontsize=14)
```

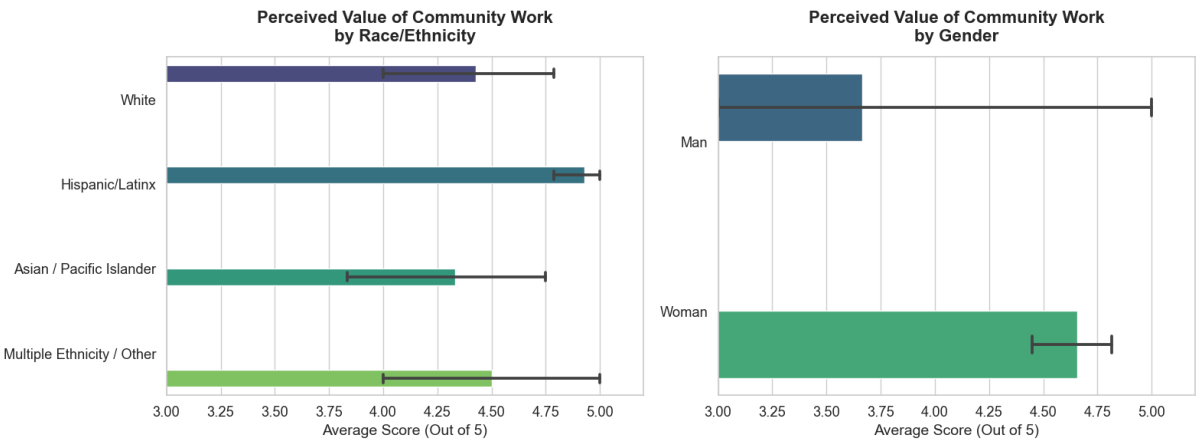
```
axes[0].set_xlabel('')
wrap_labels(axes[0], 20)

# 2. Scenario 2 Sentiment
sns.boxplot(x='major', y='scenario_2_reason_sentiment', hue='scenario_
            color='white', linewidth=1, fliersize=0, boxprops=dict(alp
sns.swarmplot(x='major', y='scenario_2_reason_sentiment', hue='scenari
              palette='viridis', marker='h', size=7, dodge=True)
axes[1].set_title('Scenario 2 Sentiment by Major\n(Community Represent
axes[1].set_ylabel('Sentiment Score', fontsize=14)
axes[1].set_xlabel('')
axes[1].legend(title='Scenario 2 Decision', bbox_to_anchor=(1.05, 1),
wrap_labels(axes[1], 20)

# 3. Scenario 5 Sentiment
sns.boxplot(x='major', y='scenario_5_reason_sentiment', hue='scenario_
            color='white', linewidth=1, fliersize=0, boxprops=dict(alp
sns.swarmplot(x='major', y='scenario_5_reason_sentiment', hue='scenari
              palette='viridis', marker='h', size=7, dodge=True)
axes[2].set_title('Scenario 5 Sentiment by Major\n(Workshop Accessibil
axes[2].set_ylabel('Sentiment Score', fontsize=14)
axes[2].set_xlabel('')
axes[2].legend(title='Scenario 5 Decision', bbox_to_anchor=(1.05, 1),
wrap_labels(axes[2], 20)

plt.tight_layout()
plt.savefig('refined_faceted_major.png', dpi=300)
plt.show()
plt.close()
```



5. Research Question 2

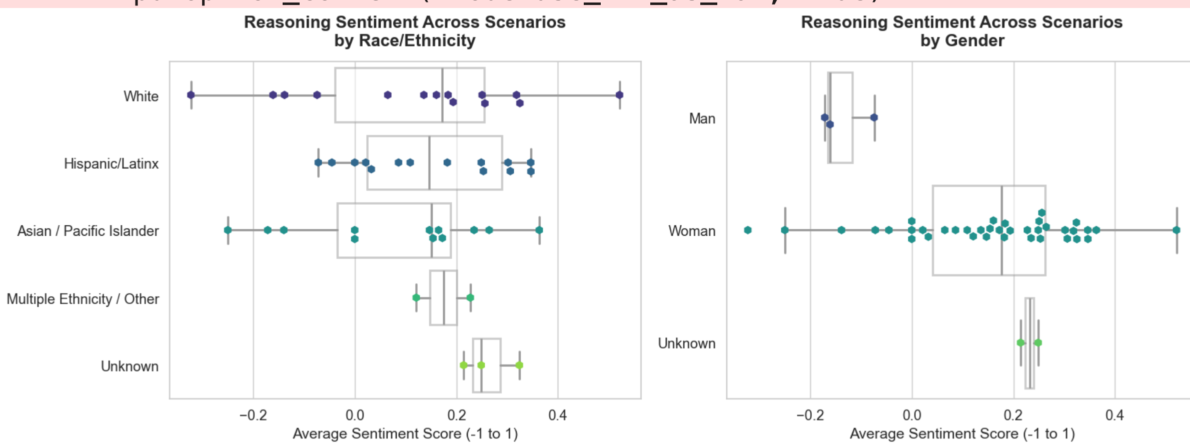
In [12]:
```
df_plot = df.dropna(subset=['ethnicity', 'gender'])
df_plot.head()
sns.set_theme(style="whitegrid", font_scale=1.2)
```

In [26]:
```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Subplot A: By Ethnicity
sns.barplot(
    data=df_plot[df_plot['ethnicity'] != 'Unknown'],
    x='community_work_valuable',
    y='ethnicity',
    hue='ethnicity',     # <-- Added to fix the warning
    ax=axes[0],
    palette='viridis',
    errorbar='ci',
    capsize=0.1,
).get_legend().remove()

axes[0].set_title('Perceived Value of Community Work\nby Race/Ethnicit
axes[0].set_xlabel('Average Score (Out of 5)', fontsize=14)
axes[0].set_ylabel('')
axes[0].set_xlim(3, 5.2)

# Subplot B: By Gender
sns.barplot(
    data=df_plot[(df_plot['gender'] != 'Prefer not to Say') &
    (df_plot['gender'] != 'Unknown')],
    x='community_work_valuable',
    y='gender',
    hue='gender',        # <-- Added to fix the warning
    ax=axes[1],
    palette='viridis',
    errorbar='ci',
    capsize=0.1
).get_legend().remove()
axes[1].set_title('Perceived Value of Community Work\nby Gender', font
axes[1].set_xlabel('Average Score (Out of 5)', fontsize=14)
axes[1].set_ylabel('')
axes[1].set_xlim(3, 5.2)

plt.tight_layout()
plt.savefig('community_work_valuable_demographics.png', dpi=300, bbox_
plt.show()
```



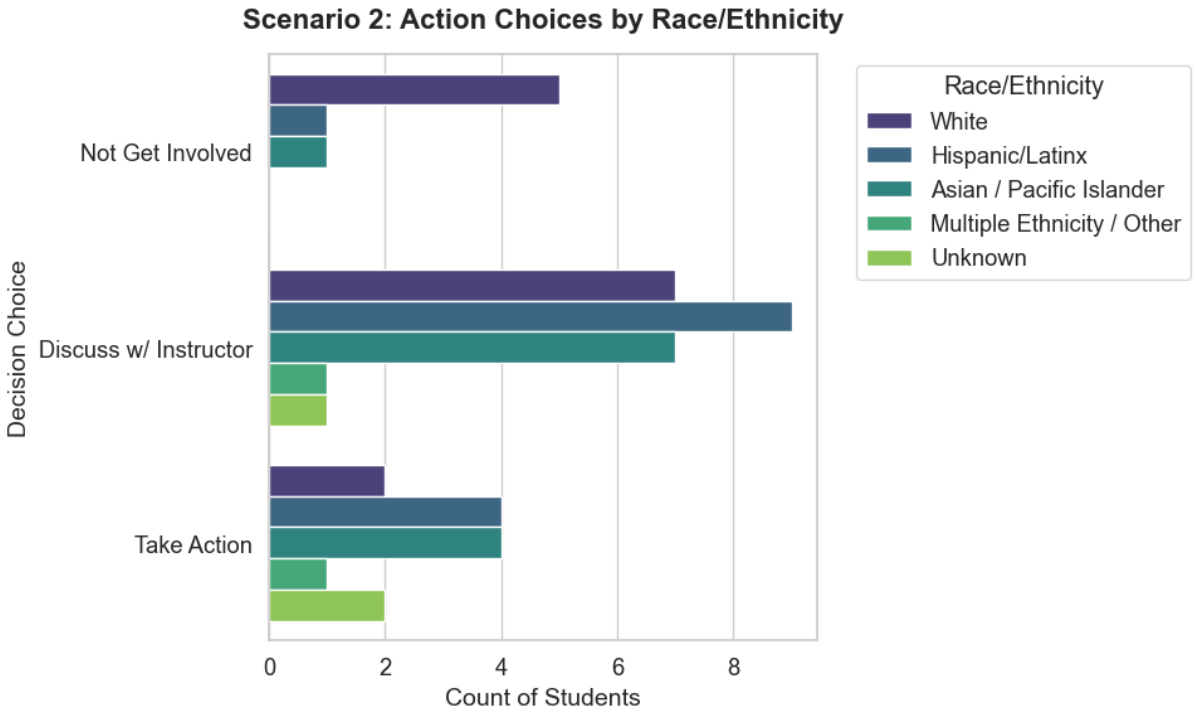
In [25]:
```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Subplot A: Sentiment by Ethnicity
sns.boxplot(
    data=df_plot,
    x='overall_scenario_sentiment',
    y='ethnicity',
    ax=axes[0],
    color='white', linewidth=2, fliersize=0, boxprops=dict(alpha=0.5)
)
sns.swarmplot(
    data=df_plot,
    x='overall_scenario_sentiment',
    y='ethnicity',
    ax=axes[0],
    palette='viridis', marker='h', size=8, hue='ethnicity', legend=Fal
)
axes[0].set_title('Reasoning Sentiment Across Scenarios\nby Race/Ethni
axes[0].set_xlabel('Average Sentiment Score (-1 to 1)', fontsize=14)
axes[0].set_ylabel('')

# Subplot B: Sentiment by Gender
sns.boxplot(
    data=df_plot[df_plot['gender'] != 'Prefer not to Say'],
    x='overall_scenario_sentiment',
    y='gender',
    ax=axes[1],
    color='white', linewidth=2, fliersize=0, boxprops=dict(alpha=0.5)
)
sns.swarmplot(
    data=df_plot[df_plot['gender'] != 'Prefer not to Say'],
    x='overall_scenario_sentiment',
    y='gender',
    ax=axes[1],
    palette='viridis', marker='h', size=8, hue='gender', legend=False
)
axes[1].set_title('Reasoning Sentiment Across Scenarios\nby Gender', f
axes[1].set_xlabel('Average Sentiment Score (-1 to 1)', fontsize=14)
axes[1].set_ylabel('')

plt.tight_layout()
plt.savefig('scenario_sentiment_demographics.png', dpi=300, bbox_inche
plt.show() # Displays the plot in the notebook
```

```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



In [26]:

```
# Map the long text answers to shorter, more readable labels
df_plot['scenario_2_short'] = df_plot['scenario_2'].map({
    'I would first discuss the situation with my instructor or the com
    'I would take some form of action to ensure the community’s concer
    'Given the circumstances, it would be better not to get involved o
})

fig, ax = plt.subplots(figsize=(10, 6))
sns.countplot(
    data=df_plot,
    y='scenario_2_short',
    hue='ethnicity',
    palette='viridis'
)
ax.set_title('Scenario 2: Action Choices by Race/Ethnicity', fontsize=
ax.set_xlabel('Count of Students', fontsize=14)
ax.set_ylabel('Decision Choice', fontsize=14)
plt.legend(title='Race/Ethnicity', bbox_to_anchor=(1.05, 1), loc='uppe

plt.tight_layout()
plt.savefig('scenario_2_action_ethnicity.png', dpi=300, bbox_inches='t
plt.show()# Displays the plot in the notebook
```



6. Research Question 3

(a) Construct Composite Scores

In [31]:
```
rq3_core = ['community_partners_inclusion', 'community_partners_unders
df['partner_value'] = df[rq3_core].mean(axis=1)

# Also create version with reverse-coded lack_of_interaction
df['lack_interaction_rev'] = 6 - df['lack_of_interaction_with_partners
rq3_full = rq3_core + ['lack_interaction_rev']
df['partner_value_full'] = df[rq3_full].mean(axis=1)
```

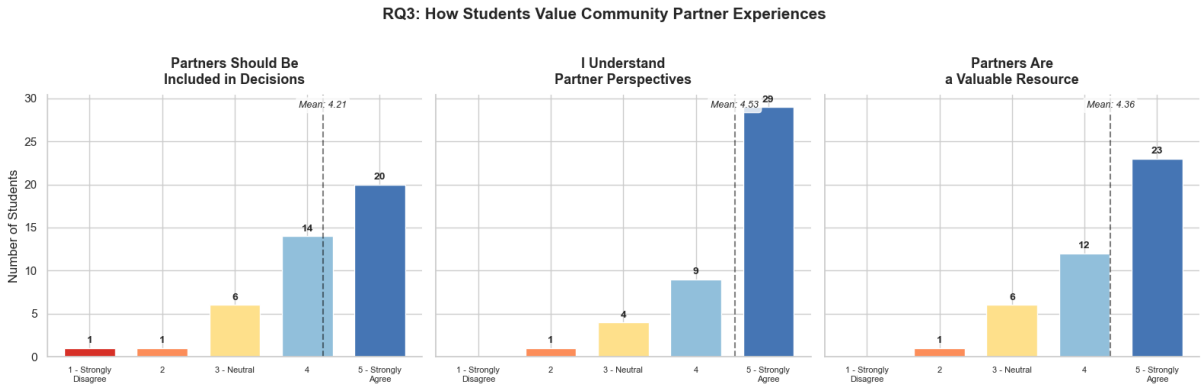
(b) Colors

In [33]:
```
PAL = {
    'Asian / Pacific Islander': '#4C72B0',
    'Hispanic/Latinx': '#DD8452',
    'White': '#55A868',
    'Multiple Ethnicity / Other': '#C44E52'
}
ETH_ORDER = ['Asian / Pacific Islander', 'Hispanic/Latinx', 'White', '
```

(c) FIGURE 1: Partner Valuation Item-Level Breakdown

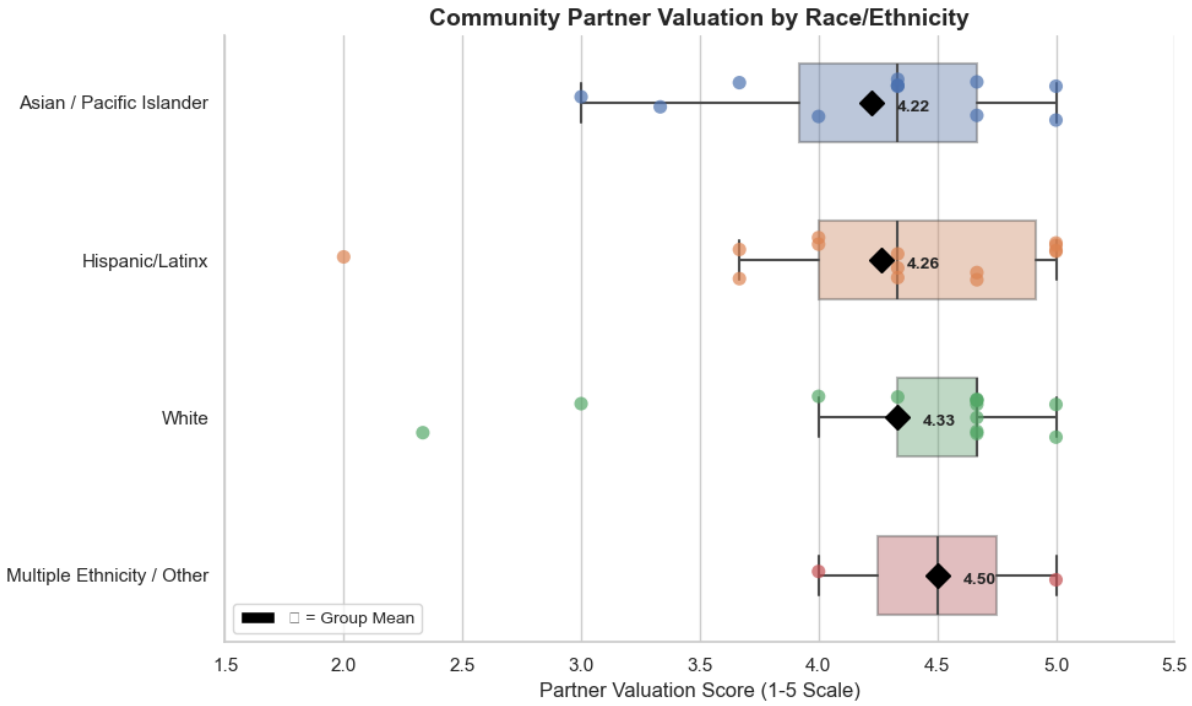
In [35]:
```
fig, axes = plt.subplots(1, 3, figsize=(16, 5), sharey=True)
item_labels = {
    'community_partners_inclusion': 'Partners Should Be\nIncluded in D
```

```
    'community_partners_understanding': 'I Understand\nPartner Perspec
    'do_patners_help': 'Partners Are\na Valuable Resource'
}

colors_likert = ['#d73027', '#fc8d59', '#fee08b', '#91bfdb', '#4575b4'
likert_labels = ['1 - Strongly\nDisagree', '2', '3 - Neutral', '4', '5

for ax, col in zip(axes, rq3_core):
    counts = df[col].dropna().value_counts().sort_index()
    # Ensure all 5 values present
    for v in [1, 2, 3, 4, 5]:
        if v not in counts.index:
            counts[v] = 0
    counts = counts.sort_index()

    bars = ax.bar(counts.index, counts.values, color=colors_likert, ed
    ax.set_title(item_labels[col], fontsize=13, fontweight='bold', pad
    ax.set_xlabel('')
    ax.set_xticks([1, 2, 3, 4, 5])
    ax.set_xticklabels(likert_labels, fontsize=8)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    # Add count labels on bars
    for bar in bars:
        h = bar.get_height()
        if h > 0:
            ax.text(bar.get_x() + bar.get_width()/2., h + 0.3, f'{int(
                    ha='center', va='bottom', fontsize=10, fontweight=

axes[0].set_ylabel('Number of Students', fontsize=12)

# Add overall means as annotation
for ax, col in zip(axes, rq3_core):
    m = df[col].mean()
    ax.axvline(x=m, color='black', linestyle='--', alpha=0.5, linewidt
    ax.text(m, ax.get_ylim()[1]*0.95, f'Mean: {m:.2f}', ha='center', f
            style='italic', bbox=dict(boxstyle='round,pad=0.3', faceco

fig.suptitle('RQ3: How Students Value Community Partner Experiences',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('rq3_item_distributions.png', dpi=150, bbox_inches='tight'
plt.show()
plt.close()
```



(d) FIGURE 2: Partner Value Composite by Ethnicity (with individual points)

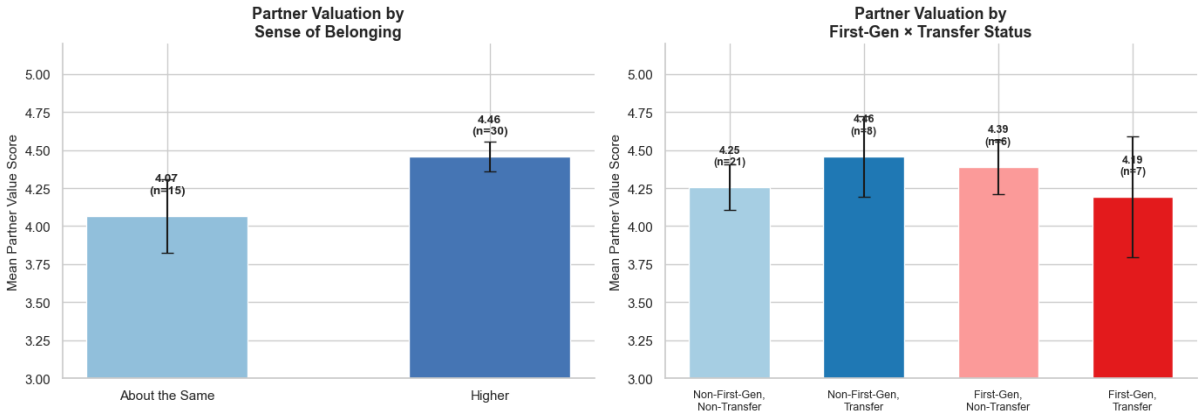
In [38]:
```
fig, ax = plt.subplots(figsize=(10, 6))

eth_data = df.dropna(subset=['ethnicity', 'partner_value'])

# Boxplot + swarmplot
sns.boxplot(data=eth_data, y='ethnicity', x='partner_value', order=ETH
            palette=PAL, width=0.5, boxprops=dict(alpha=0.4),
            fliersize=0, ax=ax, linewidth=1.5)
sns.stripplot(data=eth_data, y='ethnicity', x='partner_value', order=E
              palette=PAL, size=8, alpha=0.7, jitter=0.15, ax=ax)

# Add mean markers
for i, eth in enumerate(ETH_ORDER):
    m = eth_data[eth_data['ethnicity']==eth]['partner_value'].mean()
    ax.plot(m, i, 'D', color='black', markersize=10, zorder=5)
    ax.annotate(f'{m:.2f}', (m, i), textcoords='offset points', xytext
                fontsize=10, fontweight='bold')

ax.set_xlabel('Partner Valuation Score (1-5 Scale)', fontsize=12)
ax.set_ylabel('')
ax.set_title('Community Partner Valuation by Race/Ethnicity', fontsize
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.set_xlim(1.5, 5.5)

# Add diamond legend
diamond = mpatches.Patch(facecolor='black', label='◆ = Group Mean')
ax.legend(handles=[diamond], loc='lower left', fontsize=10)

plt.tight_layout()
plt.savefig('rq3_partner_value_by_ethnicity.png', dpi=150, bbox_inches
plt.show()
plt.close()
```



(e) FIGURE 3: Partner Value by Belonging Rate & First-Gen Status

In [40]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# A: By Belonging Rate
grp_belong = df.groupby('belonging_rate')['partner_value'].agg(['mean'
grp_belong = grp_belong.reindex(['About the same', 'Higher'])
bars = axes[0].bar(range(len(grp_belong)), grp_belong['mean'],
                   yerr=grp_belong['std']/np.sqrt(grp_belong['count'])
                   color=['#91bfdb', '#4575b4'], edgecolor='white', wi
                   capsize=5, error_kw={'linewidth': 1.5})
axes[0].set_xticks(range(len(grp_belong)))
axes[0].set_xticklabels(['About the Same', 'Higher'], fontsize=11)
axes[0].set_ylabel('Mean Partner Value Score', fontsize=11)
axes[0].set_title('Partner Valuation by\nSense of Belonging', fontsize
axes[0].set_ylim(3.0, 5.2)
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

for bar, row in zip(bars, grp_belong.itertuples()):
    axes[0].text(bar.get_x() + bar.get_width()/2., bar.get_height() +
                 f'{row.mean:.2f}\n(n={int(row.count)})', ha='center',

# B: By First-Gen x Transfer
cats = [
    ('Non-First-Gen,\nNon-Transfer', (df['first_gen_college']=='No') &
    ('Non-First-Gen,\nTransfer', (df['first_gen_college']=='No') & (df
    ('First-Gen,\nNon-Transfer', (df['first_gen_college']=='Yes') & (d
    ('First-Gen,\nTransfer', (df['first_gen_college']=='Yes') & (df['t
```

```
]

means, ses, ns = [], [], []
for label, mask in cats:
    vals = df.loc[mask, 'partner_value'].dropna()
    means.append(vals.mean())
    ses.append(vals.std() / np.sqrt(len(vals)) if len(vals) > 1 else 0
    ns.append(len(vals))

bar_colors = ['#a6cee3', '#1f78b4', '#fb9a99', '#e31a1c']
bars2 = axes[1].bar(range(4), means, yerr=ses, color=bar_colors, edgec
                    width=0.6, capsize=5, error_kw={'linewidth': 1.5})
axes[1].set_xticks(range(4))
axes[1].set_xticklabels([c[0] for c in cats], fontsize=9)
axes[1].set_ylabel('Mean Partner Value Score', fontsize=11)
axes[1].set_title('Partner Valuation by\nFirst-Gen × Transfer Status',
axes[1].set_ylim(3.0, 5.2)
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

for bar, m, n in zip(bars2, means, ns):
    axes[1].text(bar.get_x() + bar.get_width()/2., bar.get_height() +
                 f'{m:.2f}\n(n={n})', ha='center', fontsize=9, fontwei

plt.tight_layout()
plt.savefig('rq3_partner_value_belonging_firstgen.png', dpi=150, bbox_
plt.show()
plt.close()
```

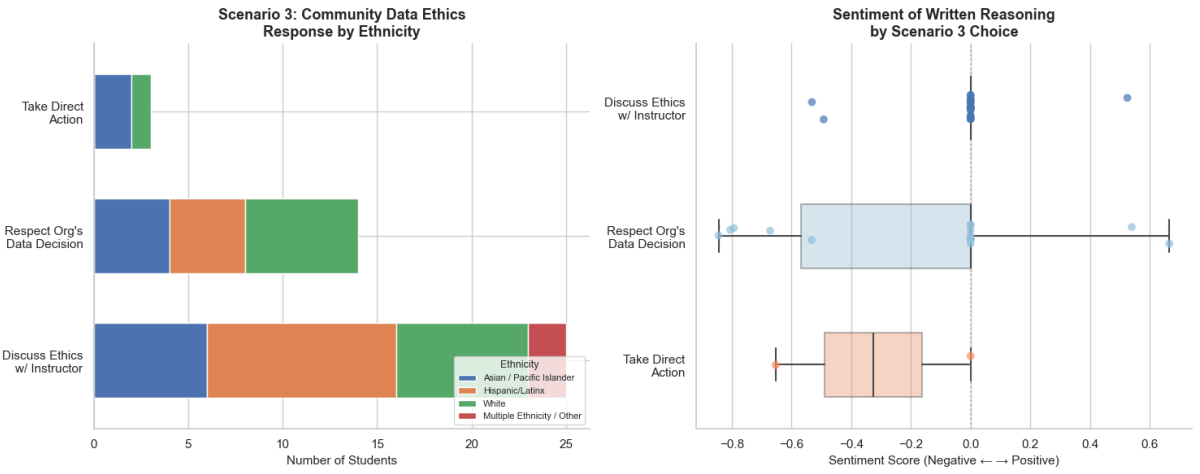


(f) FIGURE 4: Scenario 3 Analysis (Ethical Data Use - Behavioral RQ3 Measure)

In [42]:

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Short labels for scenario choices
s3_map = {
    'I would suggest discussing the ethical concerns with the instruct
    'Given the circumstances, I would agree it is better not to pursue
    'I would take some form of action related to how the data are used
```

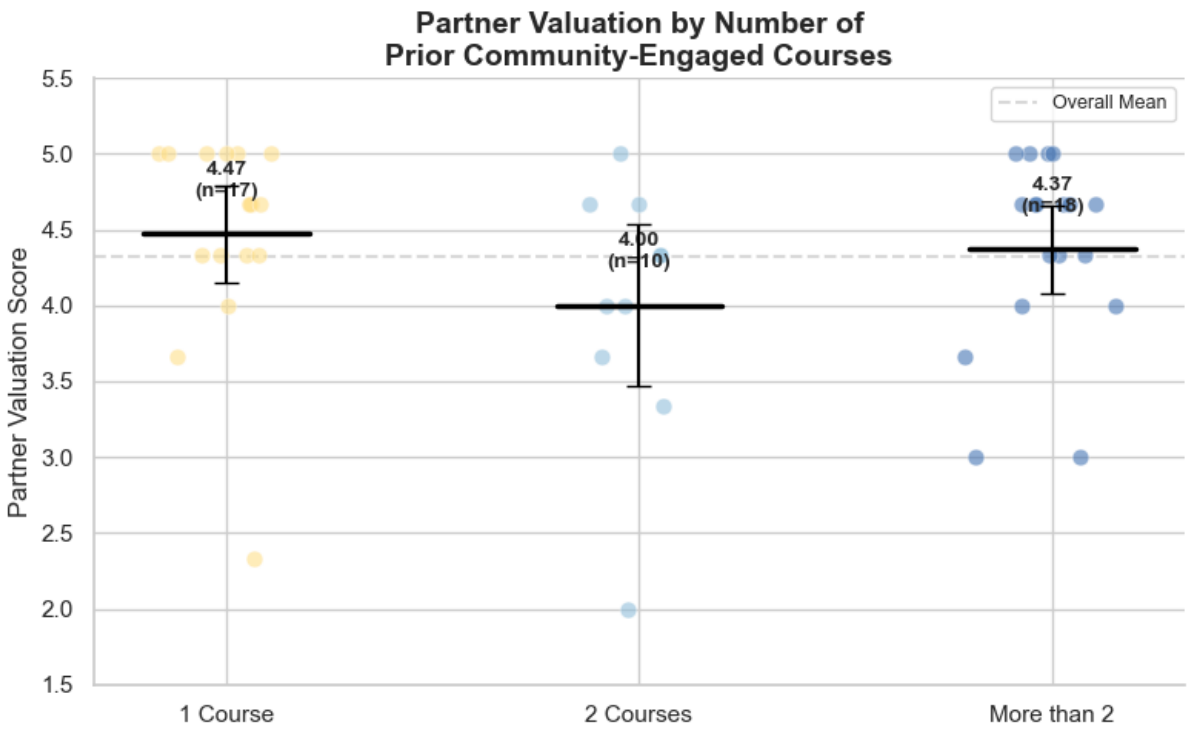
```
}
df['scenario_3_short'] = df['scenario_3'].map(s3_map)

# A: Overall distribution with ethnicity breakdown
s3_order = ['Discuss Ethics\nw/ Instructor', "Respect Org's\nData Deci
ct = pd.crosstab(df['scenario_3_short'], df['ethnicity'])
ct = ct.reindex(s3_order).fillna(0)

ct[ETH_ORDER].plot(kind='barh', stacked=True, ax=axes[0],
                   color=[PAL[e] for e in ETH_ORDER], edgecolor='whit
axes[0].set_xlabel('Number of Students', fontsize=11)
axes[0].set_ylabel('')
axes[0].set_title('Scenario 3: Community Data Ethics\nResponse by Ethn
axes[0].legend(title='Ethnicity', fontsize=8, title_fontsize=9, loc='l
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# B: Sentiment of reasoning by choice
s3_sent = df.dropna(subset=['scenario_3_short', 'scenario_3_reason_sen
sns.boxplot(data=s3_sent, y='scenario_3_short', x='scenario_3_reason_s
            order=s3_order, palette=['#4575b4', '#91bfdb', '#fc8d59'],
            width=0.5, fliersize=0, ax=axes[1], boxprops=dict(alpha=0.
sns.stripplot(data=s3_sent, y='scenario_3_short', x='scenario_3_reason
              order=s3_order, palette=['#4575b4', '#91bfdb', '#fc8d59'
              size=7, alpha=0.7, jitter=0.1, ax=axes[1])
axes[1].axvline(x=0, color='gray', linestyle=':', alpha=0.5)
axes[1].set_xlabel('Sentiment Score (Negative ← → Positive)', fontsize
axes[1].set_ylabel('')
axes[1].set_title('Sentiment of Written Reasoning\nby Scenario 3 Choic
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

plt.tight_layout()
plt.savefig('rq3_scenario3_analysis.png', dpi=150, bbox_inches='tight'
plt.show()
plt.close()
```



(g) FIGURE 5: Partner Value vs Prior XP Courses (Dosage

Effect)

In [43]:
```
fig, ax = plt.subplots(figsize=(8, 5))

xp_order = ['1', '2', 'More than 2']
xp_colors = ['#fee08b', '#91bfdb', '#4575b4']

for i, (xp, color) in enumerate(zip(xp_order, xp_colors)):
    vals = df[df['xp_courses']==xp]['partner_value'].dropna()
    # Jittered strip
    jitter = np.random.normal(i, 0.08, len(vals))
    ax.scatter(jitter, vals, c=color, s=60, alpha=0.6, edgecolors='whi
    # Mean + CI
    m = vals.mean()
    se = vals.std() / np.sqrt(len(vals))
    ax.plot([i-0.2, i+0.2], [m, m], color='black', linewidth=2.5, zord
    ax.errorbar(i, m, yerr=1.96*se, color='black', linewidth=1.5, caps
    ax.text(i, m + 0.25, f'{m:.2f}\n(n={len(vals)})', ha='center', fon

ax.set_xticks(range(3))
ax.set_xticklabels(['1 Course', '2 Courses', 'More than 2'], fontsize=
ax.set_ylabel('Partner Valuation Score', fontsize=12)
ax.set_title('Partner Valuation by Number of\nPrior Community-Engaged
ax.set_ylim(1.5, 5.5)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.axhline(y=df['partner_value'].mean(), color='gray', linestyle='--',
ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('rq3_partner_value_xp_dosage.png', dpi=150, bbox_inches='t
plt.show()
plt.close()
```


Partner Valuation by Number of
Prior Community-Engaged Courses
Partner Valuation Score
5.5
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
Overall Mean
4.47
(n=17)
4.00
(n=10)
4.37
(n=18)
1 Course
2 Courses
More than 2